

Software Requirements Specification

Project Title: CalPilot – Local AI Calendar Agent

Version: 1.0

Methodology: Agile

Prepared for: Personal / Portfolio Project

Technology Stack: React.js, FastAPI, LM Studio, Google Calendar API, SQLite/PostgreSQL

Primary Goal: Build an AI-assisted calendar event creation system that allows users to create Google Calendar events faster using presets and natural language commands.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification document is to define the functional, non-functional, technical, and Agile planning requirements for **CalPilot – Local AI Calendar Agent**.

The application will help users create Google Calendar events quickly by using predefined event templates and natural language commands. The system will integrate with Google Calendar and use a locally running AI model through LM Studio to understand user commands.

The first version will focus on a common real-world use case: creating part-time work shift events such as Tesco shifts with predefined title, shift leader, time, reminder, and color.

1.2 Project Overview

CalPilot is a full-stack web application that allows a user to create Google Calendar events faster using two methods:

1. **Preset-based event creation**

2. The user selects predefined options such as event type, shift leader, time, reminder, and color.

3. The system generates a ready-to-create calendar event preview.

4. **AI-based natural language event creation**

5. The user writes a command such as: "Add Tesco shift tomorrow 3 to 11 with FH"

6. The local AI model extracts event details.
7. The system applies saved presets and generates an event preview.
8. The user confirms the preview.
9. The event is created in Google Calendar.

The system will work like a simple AI agent. It will understand commands, apply user preferences, validate missing information, prepare the event, and create the Google Calendar event after user confirmation.

1.3 Intended Users

The primary intended user is an individual who frequently creates similar Google Calendar events.

Example users include:

- Part-time workers adding shift schedules
- Students adding classes, study sessions, and supervisor meetings
- Freelancers managing repeated client appointments
- Professionals creating regular meetings
- Any user who wants a faster way to create calendar events

For the first MVP, the application will mainly target one user: the developer/user of the system.

1.4 Scope

The system will allow the user to:

- Connect their Google Calendar account
- Create event templates/presets
- Select predefined event options
- Generate event previews
- Use natural language commands to generate event details
- Confirm and create events in Google Calendar
- Store basic preferences locally
- Use a local AI model through LM Studio
- Avoid paid AI APIs

The system will not initially support:

- Multi-user production authentication
- Payment features
- Team calendar management
- Complex recurring event rules
- Mobile app development
- Voice commands

- Advanced scheduling conflict resolution
 - Automatic background event creation without user confirmation
-

2. Overall Description

2.1 Product Perspective

CalPilot will be a personal productivity web application. It will connect a React.js frontend with a FastAPI backend. The backend will communicate with Google Calendar API and a local LM Studio AI server.

The system will follow this architecture:

User → React Frontend → FastAPI Backend → LM Studio / Google Calendar API

The AI will not directly create events. It will only extract and structure event data. The backend will validate the AI output and create the event only after the user confirms.

2.2 Product Functions

The main functions of the system are:

- Google Calendar connection
 - Event template management
 - Preset-based event generation
 - Natural language command processing
 - AI-based event data extraction
 - Event preview generation
 - Event validation
 - Google Calendar event creation
 - Reminder configuration
 - Calendar color selection
 - Basic local storage/database storage of presets
-

2.3 User Classes and Characteristics

2.3.1 Primary User

The primary user is someone who regularly creates calendar events and wants to save time.

Characteristics:

- Has a Google Calendar account
 - Creates repeated events such as work shifts
 - Wants a simple and fast interface
 - May not want to manually fill every calendar field
 - Wants to use free/local AI rather than paid APIs
-

2.3.2 Developer/User

Since this is also a portfolio project, the developer is also a user.

Characteristics:

- Wants to learn React.js and FastAPI
 - Wants to integrate Google Calendar API
 - Wants to build a practical AI agent
 - Wants to use LM Studio for local AI
 - Wants a production-style project for portfolio demonstration
-

2.4 Operating Environment

The application will run in a local development environment first.

Frontend:

- React.js
- Vite
- TailwindCSS
- Browser: Chrome, Edge, or Firefox

Backend:

- Python
- FastAPI
- Uvicorn
- Pydantic
- SQLAlchemy
- SQLite for MVP
- PostgreSQL later

AI Environment:

- LM Studio installed locally

- Local model downloaded and running
- LM Studio local API server enabled

External Service:

- Google Calendar API
 - Google OAuth 2.0
-

2.5 Design and Implementation Constraints

The application must follow these constraints:

- The project should use free tools only.
 - AI processing must use LM Studio locally.
 - The backend must be implemented using FastAPI.
 - The frontend must be implemented using React.js.
 - Google Calendar secrets must not be exposed in the frontend.
 - Calendar event creation must require user confirmation.
 - The system should initially support one main user.
 - The system should be simple enough to complete step by step.
 - The system should be designed in a way that can later be extended.
-

2.6 Assumptions and Dependencies

Assumptions:

- The user has a Google account.
- The user can create a Google Cloud project.
- The user can enable the Google Calendar API.
- The user can run LM Studio locally.
- The user has a machine capable of running a small local AI model.
- The user will confirm an event before it is created.

Dependencies:

- Google Calendar API availability
 - Google OAuth authentication
 - LM Studio local API server
 - Local machine performance
 - Browser compatibility
 - Internet connection for Google Calendar sync
-

3. System Features

3.1 Feature 1: Google Calendar Connection

Description

The user must be able to connect their Google Calendar account using Google OAuth 2.0.

Functional Requirements

- The system shall provide a "Connect Google Calendar" button.
- The system shall redirect the user to Google OAuth authentication.
- The system shall receive OAuth credentials after successful login.
- The system shall store access and refresh tokens securely in the backend.
- The system shall show whether the calendar is connected.
- The system shall use the connected account to create calendar events.

Priority

High

MVP Status

Required

3.2 Feature 2: Preset-Based Event Creation

Description

The user must be able to create events quickly using predefined templates and options.

Example:

- Event type: Shift @ Tesco
- Shift leader: FH
- Time: 3 PM to 11 PM
- Reminder: 30 minutes before
- Color: Work color

Functional Requirements

- The system shall allow the user to select an event template.
- The system shall auto-fill default title, time, reminder, and color based on the selected template.
- The system shall allow the user to select a date.
- The system shall allow the user to select a shift leader.
- The system shall generate the event title automatically.
- The system shall generate an event preview before creation.
- The system shall allow the user to confirm and create the event.

Priority

High

MVP Status

Required

3.3 Feature 3: Natural Language Event Generation

Description

The user must be able to enter natural language commands to generate event data.

Example command:

“Add Tesco shift tomorrow 3 to 11 with FH”

Functional Requirements

- The system shall provide a text input for natural language commands.
- The system shall send the command to the FastAPI backend.
- The backend shall send the command to LM Studio.
- LM Studio shall return structured JSON event data.
- The backend shall validate the returned event data.
- The system shall apply default preset values where necessary.
- The system shall return a preview to the frontend.
- The user shall review the event before creating it.

Priority

High

MVP Status

Required

3.4 Feature 4: AI Agent Event Processing

Description

The system will behave like a simple AI agent by understanding user intent, applying templates, checking missing fields, and preparing an event.

Functional Requirements

- The system shall detect whether the user wants to create a calendar event.
- The system shall identify the event type from the command.
- The system shall extract date, start time, end time, shift leader, reminder, and event title.
- The system shall apply saved preset values if some fields are missing.
- The system shall identify missing required fields.
- The system shall generate a structured event preview.
- The system shall not create an event automatically without confirmation.

Priority

High

MVP Status

Required

3.5 Feature 5: Event Preview

Description

Before creating a Google Calendar event, the system must show a clear preview.

Functional Requirements

The event preview shall display:

- Event title
- Date

- Start time
- End time
- Reminder
- Calendar color
- Event type
- Shift leader, if applicable

The user shall be able to:

- Confirm the event
- Edit event details before creation
- Cancel the event creation

Priority

High

MVP Status

Required

3.6 Feature 6: Google Calendar Event Creation

Description

After confirmation, the system must create the event in the user's Google Calendar.

Functional Requirements

- The backend shall create an event using Google Calendar API.
- The backend shall pass title, start time, end time, reminder, timezone, and color to Google Calendar.
- The backend shall return a success response after event creation.
- The frontend shall display a success message.
- The frontend shall provide a link to open the created event in Google Calendar, if available.

Priority

High

MVP Status

Required

3.7 Feature 7: Preset Management

Description

The user should be able to manage event templates.

Functional Requirements

The system shall allow the user to:

- View existing presets
- Create a new preset
- Edit a preset
- Delete a preset
- Set default title
- Set default time range
- Set default reminder
- Set default color
- Add optional custom fields such as shift leader

Priority

Medium

MVP Status

Partially required

For the first version, presets may be stored in a static JSON file or seeded database.

3.8 Feature 8: Local AI Configuration

Description

The system should allow the backend to communicate with LM Studio.

Functional Requirements

- The system shall store LM Studio API base URL in environment variables.
- The system shall call LM Studio using an OpenAI-compatible API format.
- The system shall handle timeout errors.
- The system shall handle invalid AI responses.

- The system shall parse AI output into validated JSON.
- The system shall return meaningful error messages if AI generation fails.

Priority

High

MVP Status

Required

4. External Interface Requirements

4.1 User Interface Requirements

The frontend should be clean, simple, and responsive.

Main pages:

1. Dashboard
2. Create Event
3. Presets/Settings

Dashboard UI

The dashboard shall display:

- App name
- Google Calendar connection status
- Quick command box
- Recent generated event previews
- Quick access to Tesco shift creation

Create Event UI

The Create Event page shall include:

- Natural language command input
- Generate Preview button
- Manual preset form
- Event preview card
- Confirm & Create Event button

Presets/Settings UI

The settings page shall include:

- List of event templates
 - Add template button
 - Edit template form
 - Delete option
 - Default reminder setting
 - Default color setting
-

4.2 Backend API Interface Requirements

The backend shall provide REST API endpoints.

Main endpoint groups:

- /api/health
- /api/auth
- /api/calendar
- /api/ai
- /api/presets

Example endpoints:

- GET /api/health
 - GET /api/auth/google
 - GET /api/auth/google/callback
 - POST /api/ai/generate-event
 - POST /api/calendar/events
 - GET /api/presets
 - POST /api/presets
 - PUT /api/presets/{preset_id}
 - DELETE /api/presets/{preset_id}
-

4.3 Google Calendar API Interface

The system shall use Google Calendar API to create events.

Required event fields:

- Summary/title
- Start date and time

- End date and time
- Timezone
- Reminder
- Color ID

The default timezone shall be:

Europe/London

4.4 LM Studio Interface

The backend shall communicate with LM Studio using HTTP requests.

Default local URL:

http://localhost:1234/v1/chat/completions

The backend shall send:

- System prompt
- User command
- Known presets
- Current date
- Timezone
- Required JSON schema

The backend shall receive:

- AI-generated structured JSON event data
-

5. Functional Requirements

FR-001: Connect Google Calendar

The system shall allow the user to connect their Google Calendar account through Google OAuth.

Priority: High

FR-002: Show Calendar Connection Status

The system shall display whether Google Calendar is connected.

Priority: High

FR-003: Create Event Using Preset

The system shall allow the user to create a calendar event using predefined options.

Priority: High

FR-004: Generate Event Title Automatically

The system shall generate event titles based on selected preset and fields.

Example:

Shift @ Tesco - FH

Priority: High

FR-005: Select Event Date

The system shall allow the user to select the event date.

Priority: High

FR-006: Select Event Time

The system shall allow the user to select or edit start and end time.

Priority: High

FR-007: Select Reminder

The system shall allow the user to select reminder timing.

Priority: High

FR-008: Select Calendar Color

The system shall allow the user to select event color.

Priority: Medium

FR-009: Generate Event from Natural Language

The system shall allow the user to generate event data from a natural language command.

Priority: High

FR-010: Validate AI Output

The system shall validate AI-generated event data before showing it to the user.

Priority: High

FR-011: Show Missing Fields

The system shall show missing required fields if the AI cannot identify all required event information.

Priority: High

FR-012: Show Event Preview

The system shall show a preview before event creation.

Priority: High

FR-013: Edit Event Preview

The system shall allow the user to edit event preview details before creation.

Priority: Medium

FR-014: Confirm Event Creation

The system shall require the user to confirm before creating the Google Calendar event.

Priority: High

FR-015: Create Google Calendar Event

The system shall create the confirmed event in Google Calendar.

Priority: High

FR-016: Show Success Message

The system shall show a success message after event creation.

Priority: High

FR-017: Manage Presets

The system shall allow the user to create, update, view, and delete presets.

Priority: Medium

FR-018: Store Presets Locally

The system shall store presets in a local database.

Priority: Medium

FR-019: Handle AI Server Errors

The system shall display an error message if LM Studio is not running or does not respond.

Priority: High

FR-020: Handle Google Calendar API Errors

The system shall display an error message if event creation fails.

Priority: High

6. Non-Functional Requirements

NFR-001: Usability

The system shall be simple and easy to use.

The user should be able to create a common shift event in fewer than 5 clicks using presets.

NFR-002: Performance

The system shall generate preset-based previews in less than 1 second.

AI-based event generation should ideally return a response within 10 seconds, depending on the local AI model and hardware.

NFR-003: Security

The system shall not expose Google API credentials in the frontend.

OAuth tokens shall be stored only in the backend.

Environment secrets shall be stored in `.env` files and excluded from Git.

NFR-004: Reliability

The system shall not create events unless the user confirms.

The system shall handle invalid AI responses gracefully.

The system shall show clear error messages for failures.

NFR-005: Maintainability

The system shall use a clean project structure.

Frontend and backend logic shall be separated.

AI logic, Google Calendar logic, and preset logic shall be implemented in separate service files.

NFR-006: Extensibility

The system shall be designed so that future features can be added, such as:

- Recurring events
 - Multiple calendars
 - Voice commands
 - Weekly shift planner
 - Event update/delete
 - Multi-user support
 - PostgreSQL deployment
 - Docker deployment
-

NFR-007: Cost

The system shall use free tools for the MVP.

The AI model shall run locally through LM Studio.

No paid OpenAI, Claude, Gemini, or other paid AI API shall be required.

NFR-008: Compatibility

The frontend shall work on modern web browsers.

The backend shall run locally on Windows, macOS, or Linux.

7. Data Requirements

7.1 Main Entities

The system will use the following main data entities:

1. User
 2. GoogleToken
 3. EventPreset
 4. ShiftLeader
 5. EventHistory
-

7.2 User Entity

Fields:

- id
- name
- email
- created_at
- updated_at

For the MVP, the system may only support one user.

7.3 GoogleToken Entity

Fields:

- id
 - user_id
 - access_token
 - refresh_token
 - token_expiry
 - scope
 - created_at
 - updated_at
-

7.4 EventPreset Entity

Fields:

- id
- name
- default_title

- default_start_time
- default_end_time
- default_reminder_minutes
- default_color_id
- created_at
- updated_at

Example:

- name: Tesco Shift
 - default_title: Shift @ Tesco
 - default_start_time: 15:00
 - default_end_time: 23:00
 - default_reminder_minutes: 30
-

7.5 ShiftLeader Entity

Fields:

- id
- short_code
- display_name
- active_status
- created_at
- updated_at

Example:

- short_code: FH
 - short_code: PK
-

7.6 EventHistory Entity

Fields:

- id
- title
- date
- start_time
- end_time
- reminder_minutes
- color_id
- google_event_id
- html_link
- created_at

This entity may be added after the first MVP.

8. AI Requirements

8.1 AI Purpose

The AI will convert natural language commands into structured event data.

The AI shall not directly create events.

8.2 AI Input

Example input:

"Add Tesco shift tomorrow 3 to 11 with FH"

8.3 AI Output

The AI must return JSON in the following structure:

```
{
  "intent": "create_calendar_event",
  "event_type": "tesco_shift",
  "title": "Shift @ Tesco - FH",
  "date": "2026-06-11",
  "start_time": "15:00",
  "end_time": "23:00",
  "shift_leader": "FH",
  "reminder_minutes": 30,
  "color_id": "9",
  "missing_fields": []
}
```

8.4 AI Validation Rules

The backend shall validate:

- Intent must be `create_calendar_event`
 - Date must be valid
 - Start time must be valid
 - End time must be valid
 - End time must be after start time
 - Title must not be empty
 - Reminder must be a number
 - Missing fields must be shown if required data is incomplete
-

8.5 AI Error Handling

The system shall handle:

- LM Studio not running
 - AI response timeout
 - Invalid JSON
 - Missing required fields
 - Conflicting information
 - Unsupported event type
-

9. User Stories

Epic 1: Calendar Connection

User Story 1.1

As a user, I want to connect my Google Calendar so that the app can create events in my calendar.

Acceptance Criteria:

- User can click "Connect Google Calendar"
 - User is redirected to Google login
 - User grants permission
 - App shows calendar as connected
-

Epic 2: Preset-Based Event Creation

User Story 2.1

As a user, I want to select a Tesco shift preset so that I do not need to type the same event details every time.

Acceptance Criteria:

- User can select Tesco Shift
 - Title is auto-filled
 - Default time is auto-filled
 - Default reminder is auto-filled
 - Default color is auto-filled
-

User Story 2.2

As a user, I want to select my shift leader short form so that the event title includes it.

Acceptance Criteria:

- User can select FH or PK
 - Title updates to `Shift @ Tesco - FH` or `Shift @ Tesco - PK`
-

User Story 2.3

As a user, I want to preview the event before creating it so that I can check if the details are correct.

Acceptance Criteria:

- Preview shows title, date, time, reminder, and color
 - User can confirm or cancel
-

Epic 3: AI Event Generation

User Story 3.1

As a user, I want to type a natural language command so that the system can generate event details automatically.

Acceptance Criteria:

- User can enter a command
 - System sends command to FastAPI
 - FastAPI sends command to LM Studio
 - AI returns structured data
 - Frontend displays event preview
-

User Story 3.2

As a user, I want the system to use my default shift settings so that I can give shorter commands.

Example:

“Add Tesco shift tomorrow with FH”

Acceptance Criteria:

- System identifies Tesco shift
 - System applies default time
 - System applies default reminder
 - System generates preview
-

Epic 4: Google Calendar Event Creation

User Story 4.1

As a user, I want to click confirm so that the event is added to my Google Calendar.

Acceptance Criteria:

- User clicks Confirm & Create
 - Backend creates event in Google Calendar
 - Success message is displayed
 - Google Calendar event link is shown if available
-

Epic 5: Preset Management

User Story 5.1

As a user, I want to manage event presets so that I can customise the system for different event types.

Acceptance Criteria:

- User can view presets
 - User can create presets
 - User can update presets
 - User can delete presets
-

10. Agile Development Plan

10.1 Agile Method

The project will follow Agile development using short iterative sprints.

Each sprint will include:

- Planning
- Implementation
- Testing
- Review
- Improvement

The project will be developed in small working increments instead of building everything at once.

10.2 Product Backlog

ID	Backlog Item	Priority
PB-001	Create project structure	High
PB-002	Create FastAPI backend skeleton	High
PB-003	Create React frontend skeleton	High
PB-004	Connect React with FastAPI	High
PB-005	Create preset event form	High
PB-006	Generate event preview	High
PB-007	Add LM Studio integration	High
PB-008	Generate event from natural language	High
PB-009	Add Google OAuth	High

ID	Backlog Item	Priority
PB-010	Create Google Calendar event	High
PB-011	Add preset management	Medium
PB-012	Add event history	Medium
PB-013	Improve UI/UX	Medium
PB-014	Add validation and error handling	High
PB-015	Add README and documentation	Medium

10.3 Sprint Plan

Sprint 1: Project Setup and Basic Architecture

Goal:

Set up the frontend and backend project structure.

Tasks:

- Create project folder
- Set up React with Vite
- Set up FastAPI backend
- Add CORS configuration
- Create health check endpoint
- Connect React frontend to backend
- Create basic dashboard layout

Deliverables:

- React app running
 - FastAPI app running
 - Frontend can call backend health endpoint
-

Sprint 2: Preset-Based Event Preview

Goal:

Build the first usable version without AI and without Google Calendar.

Tasks:

- Create event preset data
- Build preset selector
- Build date selector
- Build shift leader selector
- Build time selector
- Build reminder selector
- Generate event title
- Show event preview card

Deliverables:

- User can create a Tesco shift preview manually
 - No Google Calendar sync yet
-

Sprint 3: LM Studio AI Integration

Goal:

Use local AI to generate event previews from natural language.

Tasks:

- Start LM Studio local server
- Create FastAPI LM Studio service
- Create AI command endpoint
- Create strict AI prompt
- Parse AI JSON response
- Validate AI output
- Show AI-generated preview in React

Deliverables:

- User can type “Add Tesco shift tomorrow 3 to 11 with FH”
 - System generates event preview
-

Sprint 4: Google Calendar Integration

Goal:

Allow the app to create events in Google Calendar.

Tasks:

- Create Google Cloud project
- Enable Google Calendar API
- Configure OAuth consent screen
- Create OAuth credentials
- Add Google OAuth route in FastAPI
- Store tokens locally
- Create Google Calendar event service
- Add Confirm & Create button

Deliverables:

- User can connect Google Calendar
- User can create an event from the preview

Sprint 5: Preset Management and Agent Improvements

Goal:

Improve the AI agent and allow custom preset management.

Tasks:

- Add SQLite database
- Add SQLAlchemy models
- Store presets in database
- Create CRUD endpoints for presets
- Add preset management UI
- Improve missing field handling
- Add better validation
- Add error handling

Deliverables:

- User can manage presets
- AI agent can apply stored presets

Sprint 6: Polish, Testing, and Portfolio Preparation

Goal:

Make the project presentable as a portfolio project.

Tasks:

- Improve UI design
- Add loading states
- Add toast notifications
- Add error messages
- Add README
- Add screenshots
- Add architecture diagram
- Add demo instructions
- Test full workflow

Deliverables:

- Portfolio-ready project
 - Working local demo
 - Clear documentation
-

11. Acceptance Criteria for MVP

The MVP will be considered complete when:

- React frontend runs successfully
 - FastAPI backend runs successfully
 - User can generate a Tesco shift event preview manually
 - User can generate an event preview using AI command
 - LM Studio is integrated locally
 - Google Calendar account can be connected
 - User can confirm and create an event in Google Calendar
 - Event appears correctly in Google Calendar
 - Basic error handling is implemented
 - Project has a clear README
-

12. System Workflow

12.1 Preset Workflow

1. User opens Create Event page
2. User selects Tesco Shift preset
3. User selects date
4. User selects shift leader
5. System fills time, reminder, title, and color
6. System displays preview

7. User confirms
 8. Backend creates Google Calendar event
 9. User sees success message
-

12.2 AI Workflow

1. User enters natural language command
 2. React sends command to FastAPI
 3. FastAPI sends prompt to LM Studio
 4. LM Studio returns JSON event data
 5. FastAPI validates response
 6. FastAPI applies defaults if necessary
 7. React displays preview
 8. User confirms
 9. FastAPI creates Google Calendar event
 10. User sees success message
-

13. Risks and Mitigation

Risk	Impact	Mitigation
LM Studio returns invalid JSON	High	Add strict prompt and JSON validation
Google OAuth setup becomes confusing	Medium	Implement step by step after local preview works
Tokens exposed accidentally	High	Keep secrets only in backend <code>.env</code>
Local AI model is slow	Medium	Use small model first
User creates wrong event	Medium	Always show preview before create
Calendar API error	Medium	Add clear error messages
Scope becomes too large	High	Focus first on Tesco shift MVP

14. Future Enhancements

Future versions may include:

- Event update and delete
- Recurring event creation
- Weekly shift planner

- Drag-and-drop weekly schedule
 - Voice command support
 - Multiple Google calendars
 - Conflict detection
 - Smart suggestions based on event history
 - Mobile PWA version
 - Multi-user support
 - Docker deployment
 - PostgreSQL production database
 - GitHub Actions CI/CD
-

15. Definition of Done

A feature is considered done when:

- It is implemented
 - It works locally
 - It is connected to the frontend if required
 - It has basic validation
 - It handles obvious errors
 - It has been manually tested
 - Code is committed to Git
 - The feature does not break previous functionality
-

16. Final MVP Statement

The first version of CalPilot will be a local AI-powered calendar assistant that allows the user to create Google Calendar events using saved presets and natural language commands. It will use React.js for the frontend, FastAPI for the backend, LM Studio for local AI processing, and Google Calendar API for event creation.

The main MVP use case will be:

“Create Tesco shift events quickly by selecting presets or typing commands such as ‘Add Tesco shift tomorrow 3 to 11 with FH’.”